The Call Stack in JavaScript

How JavaScript keeps track of where it is in your code — one function call at a time.

What Is the Call Stack?

The **call stack** is a data structure that tracks the execution of function calls in JavaScript.

It follows a **Last In, First Out (LIFO)** principle — the last function pushed onto the stack is the first one to be removed when it returns.

JavaScript is **single-threaded**: only one thing runs at a time, and the call stack is how it manages that.

LIFO: Last In, First Out

Push

When a function is called, it is **pushed** onto the top of the stack.

Execute

JavaScript runs the function at the **top** of the stack.

Pop

When the function returns, it is **popped** off the stack.

This cycle repeats for every function call in your program.

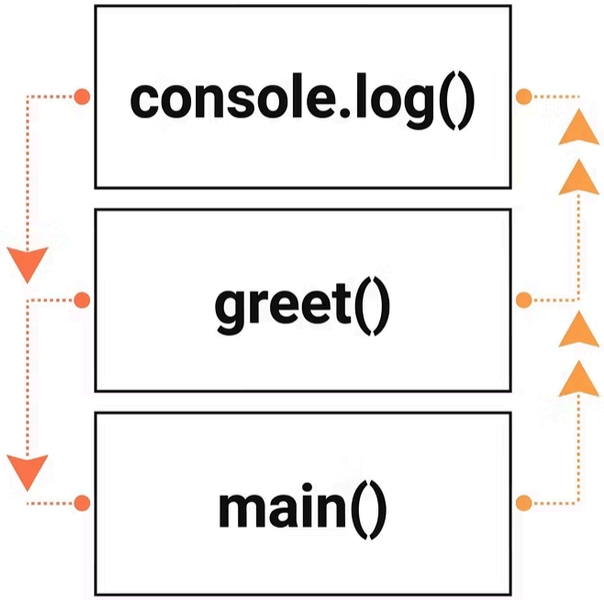
A Simple Example

```
function greet() {
 console.log("Hello!");
}

function main() {
 greet();
}

main();
```

Visualizing the Stack



Frame by Frame

1. **main()** is called → pushed to stack
2. **greet()** is called inside main → pushed on top
3. **console.log()** runs → pushed, then immediately popped
4. **greet()** finishes → popped
5. **main()** finishes → popped

Stack Overflow

What Happens?

If a function calls itself **infinitely** with no base case, the stack keeps growing until it exceeds its memory limit.

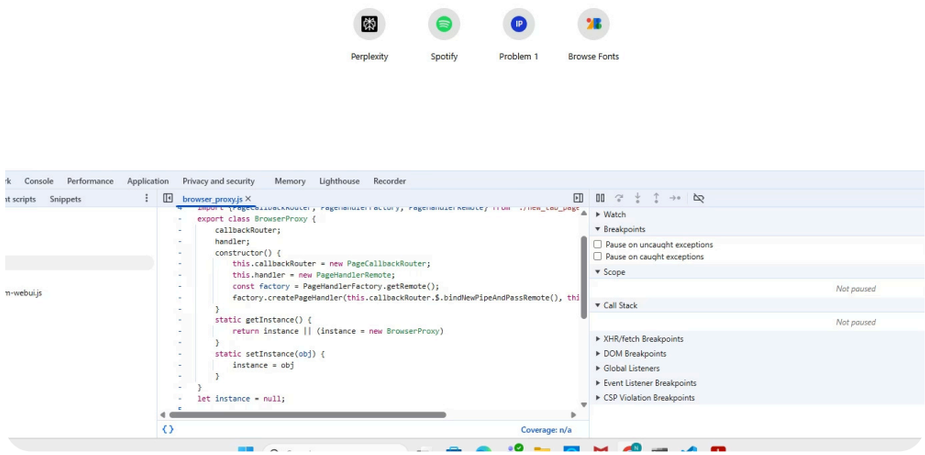
```
function loop() {
 return loop();
}
loop(); //
```

Visualizing the Call Stack

```
function one() {
  return 1;
}

function two() {
  return one() + one();
}

function three() {
  let ans = two() + one();
  console.log(ans);
}
```



JavaScript is Single-Threaded

JavaScript operates on a single-threaded execution model. This means that it has exactly one call stack and one memory heap, allowing the engine to execute only one piece of code or operation at any given time.

Because of this architecture, JavaScript cannot run two separate blocks of code simultaneously. This creates a synchronous execution flow where tasks are processed sequentially.

The primary implication of this design is that long-running tasks can "block" the main thread. When the engine is busy performing a heavy computation, it cannot respond to user interactions or other events, leading to a frozen interface until the task completes.

Workarounds for Blocking

JavaScript overcomes this limitation using the **Event Loop** and **Web APIs** (in browsers). By offloading asynchronous operations—such as network requests, timers, or file I/O—to the environment, JavaScript can continue executing other code while waiting for those background tasks to finish, maintaining non-blocking performance.

```
setTimeout(() => {
  console.log("hello world");
}, 2000);

console.log("hello...");
```

Callback Hell

Callback Hell refers to the situation where asynchronous code becomes deeply nested, making it significantly harder to read, debug, and maintain.

```javascript
h1 = document.querySelector("h1");


setTimeout(() => {
  h1.style.color = "red";
}, 1000);

setTimeout(() => {
  h1.style.color = "orange";
}, 2000);

setTimeout(() => {
  h1.style.color = "green";
}, 3000);
```

```javascript
function changeColor(color, delay) {
  setTimeout(() => {
    h1.style.color = color;
  }, delay);
}

changeColor("red", 1000);
changeColor("orange", 1000);
changeColor("green", 1000);
```

```javascript
function changeColor(color, delay, nextColorChange) {
  setTimeout(() => {
    h1.style.color = color;
    if (nextColorChange) nextColorChange();
  }, delay);
}

changeColor("red", 1000, () => {
  changeColor("orange", 1000, () => {
    changeColor("green", 1000, () => {
      changeColor("yellow", 1000, () => {
        changeColor("blue", 1000);
      });
    });
  });
});
```

```javascript
getData(function(a) {
  getMoreData(a, function(b) {
    getEvenMoreData(b, function(c) {
      getFinalData(c, function(d) {
        console.log(d);
      });
    });
  });
});
```

The Modern Solution

Modern JavaScript has effectively addressed this issue by introducing **Promises** and the **async/await** syntax, which allow developers to write asynchronous code that reads like synchronous code, avoiding deep nesting entirely.

Error uploading image.

```javascript
function saveToDb(data, success, failure) {
    let internetSpeed = Math.floor(Math.random() * 10) + 1;

    if (internetSpeed > 4) {
        success();
    } else {
        failure();
    }
}

saveToDb(
    "OpqTech",
    () => {
        console.log("success : your data was saved");
    },
    () => {
        console.log("failure: weak connection. data not saved");
    }
);
```

```javascript
savetoDb(
    "hello world",
    () => {
        console.log("success2: data2 saved");
    },
    () => {
        console.log("failure2 : weak connection");
    }
);
```

Promises

A Promise is an object representing the eventual completion or failure of an asynchronous operation. It serves as a cleaner, more robust alternative to traditional callbacks, allowing for better handling of asynchronous flows.

Pending

The initial state; the asynchronous operation has not yet completed or failed.

Fullfill

The operation completed successfully, and the promise now holds a resulting value.

Reject

The operation failed, and the promise holds the reason for the failure (an error).

```
function savetoDb(data) {
  return new Promise((success, failure) => {
    let internetSpeed = Math.floor(Math.random() * 10) + 1;
    if (internetSpeed > 4) {
      success();
    } else {
      failure();
    }
  });
}
```

JavaScript

```
let request = saveToDBPromise("OpqTech");

request
  .then(() => {
    console.log("promise resolved");
  })
  .catch(() => {
    console.log("promise rejected");
  });
```

Promise Chaining

```javascript
function savetoDb(data) {
    return new Promise((resolve, reject) => {
        let internetSpeed = Math.floor(Math.random() * 10) + 1;

        if (internetSpeed > 4) {
            resolve("success: data was saved");
        } else {
            reject("failure: weak connection");
        }
    });
}

savetoDb("OPqTech")
    .then(() => {
        console.log("data1 saved");
        return savetoDb("helloworld");
    })
    .then(() => {
        console.log("data2 saved");
        return savetoDb("shradha");
    })
    .then(() => {
        console.log("data3 saved");
    })
    .catch(() => {
        console.log("promise was rejected");
    });
```

```javascript
savetoDb("OPqTech")
  .then((result) => {
    console.log("data1 saved");
    console.log("result of promise: ", result);
    return savetoDb("helloworld");
  })
  .then((result) => {
    console.log("data2 saved");
    console.log("result of promise: ", result);
    return savetoDb("javascript");
  })
  .then((result) => {
    console.log("data3 saved");
    console.log("result of promise: ", result);
  })
  .catch((error) => {
    console.log("promise was rejected");
    console.log("error of promise: ", error);
  });
```

```javascript
h1 = document.querySelector("h1");

function changeColor(color, delay) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      h1.style.color = color;
      resolve("color changed!");
    }, delay);
  });
}

changeColor("red", 1000)
  .then(() => {
    console.log("red color was completed");
    return changeColor("orange", 1000);
  })
  .then(() => {
    console.log("orange color was completed");
    return changeColor("green", 1000);
  })
  .then(() => {
    console.log("green color was completed");
    return changeColor("blue", 1000);
  })
  .then(() => {
    console.log("blue color was completed");
  });
```